

Day 12 — HTTP, HTTPS & Complete Web Communication

Today we connect almost everything you've learned so far:

```
DNS
↓
IP
↓
Routing
↓
ARP
↓
TCP
↓
TLS
↓
HTTP/HTTPS
↓
Web Server
```

The main interview question for today is:

"What exactly happens when you type `https://example.com` in a browser and press Enter?"

By the end of Day 12, you should be able to explain that flow layer-by-layer.

Day 12 Goals

You will learn:

1. HTTP
2. HTTPS
3. HTTP request/response
4. TCP connection
5. TCP 3-way handshake
6. TLS handshake
7. HTTP over TLS
8. HTTP methods
9. HTTP status codes
10. HTTP headers
11. Cookies
12. Keep-Alive / persistent connections

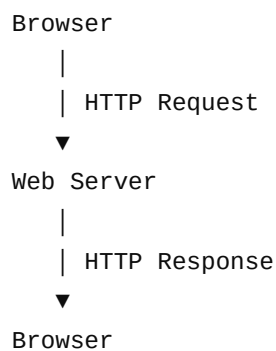
13. HTTP/1.1 vs HTTP/2 vs HTTP/3
 14. Complete browser-to-server flow
 15. Linux tools: `curl` , `ss` , `tcpdump`
 16. Interview questions
-

1. What is HTTP?

HTTP = HyperText Transfer Protocol

HTTP is an **application-layer protocol** used for communication between clients and servers.

Example:



The browser is usually the client.

The web server is usually the server.

2. Simple HTTP Example

Suppose you request:

```
http://example.com/
```

The browser might send something conceptually like:

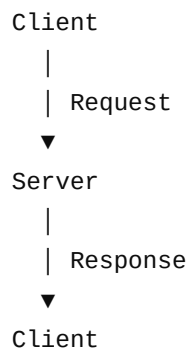
```
GET / HTTP/1.1
Host: example.com
```

The server responds with something like:

```
HTTP/1.1 200 OK
Content-Type: text/html
```

```
<html>
...
</html>
```

So HTTP has two major directions:

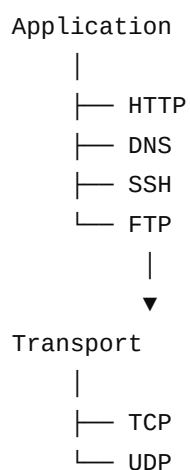


3. HTTP Is Layer 7

From your OSI learning:

```
Layer 7 → HTTP
Layer 6 → Presentation concepts
Layer 5 → Session concepts
Layer 4 → TCP/UDP
Layer 3 → IP
Layer 2 → Ethernet/Wi-Fi
Layer 1 → Physical
```

In the common TCP/IP model:





4. HTTP vs HTTPS

You will see:

`http://`

and:

`https://`

The important difference is security.

HTTP

HTTP
↓
TCP
↓
IP

HTTPS

Conceptually:

HTTP
↓
TLS
↓
TCP
↓
IP

TLS provides cryptographic protection for the HTTP communication.

5. Why Do We Need HTTPS?

Imagine sending:

```
username=deepak  
password=12345
```

without encryption.

Someone capable of observing the traffic could potentially see sensitive information.

HTTPS protects the application data using TLS.

Conceptually:

```
HTTP data  
  ↓  
TLS encryption/protection  
  ↓  
TCP  
  ↓  
IP  
  ↓  
Network
```

The network can still see important routing metadata such as IP addresses, but the HTTP payload is protected by TLS.

6. Important: HTTPS Does NOT Mean "No TCP"

For traditional HTTPS over HTTP/1.1 or HTTP/2:

```
HTTPS  
  ↓  
TLS  
  ↓  
TCP  
  ↓  
IP
```

So HTTPS normally still uses TCP in those versions.

HTTP/3 is different and uses QUIC over UDP.

We'll cover that later.

7. What Happens When You Type a Website?

Suppose you type:

```
https://example.com
```

Let's build the complete flow.

Step 1 — Browser Parses URL

The browser identifies:

```
Protocol = HTTPS
Hostname = example.com
Port = 443
Path = /
```

So:

```
https://example.com/
```

becomes conceptually:

```
Protocol → HTTPS
Host      → example.com
Port      → 443
Path      → /
```

8. Step 2 — DNS Resolution

The browser/system needs an IP address.

```
example.com
  ↓
DNS
  ↓
IP address
```

For example:

```
example.com
  ↓
93.184.x.x
```

The exact address can change and may differ by DNS resolver/location.

9. Step 3 — Routing Decision

Now Linux knows:

Destination IP

The kernel checks:

ip route

Conceptually:

```
Destination IP
  ↓
Routing table
  ↓
Which interface?
  ↓
Which next hop?
```

For a remote server:

```
PC
  ↓
Default Gateway
  ↓
Router
```

10. Step 4 — ARP / Neighbor Resolution

If the next hop is on Ethernet/IPv4, Linux needs the next-hop MAC address.

Conceptually:

Who has 192.168.1.1?

Gateway:

```
192.168.1.1
  ↓
```



MAC address

Linux can inspect neighbor information with:

```
ip neigh
```

Then Ethernet can be constructed:

```
Source MAC
Destination MAC
    ↓
Ethernet Frame
```

11. Step 5 — TCP Connection

For traditional HTTPS using TCP:

```
Client
  |
  | SYN
  ▼
Server
  |
  | SYN + ACK
  ▼
Client
  |
  | ACK
  ▼
Server
```

This is the TCP 3-way handshake you learned earlier.

12. TCP Handshake Again

Let's make it very simple.

1. SYN

Client says:

"I want to establish a TCP connection."

PC — SYN —> Server

2. SYN + ACK

Server says:

"I received your request, and I also want to establish the connection."

PC ← SYN + ACK — Server

3. ACK

Client says:

"I received your response."

PC — ACK —> Server

Now:

TCP connection established

13. Important Interview Point

The TCP handshake does **not** contain your HTTP request.

First:

SYN
SYN+ACK
ACK

Then HTTP data can be exchanged.

TCP handshake
↓
TCP connection established
↓
HTTP request

14. Step 6 — TLS Handshake

Because we're using:

https://

TCP isn't enough.

The client and server need to establish a secure TLS session.

Conceptually:

```
TCP connection
  ↓
TLS handshake
  ↓
Secure TLS session
  ↓
HTTP request
```

15. What Does TLS Do?

TLS provides important security properties:

Confidentiality

Others should not be able to read the protected application data.

Integrity

The data should not be silently modified without detection.

Authentication

The client can authenticate the server using certificates and the public-key infrastructure.

16. TLS Certificate

During TLS setup, the server provides certificate information.

Conceptually:

```
Server
  |
  | Certificate
  ▼
Client
```

The certificate contains information binding the server's identity to a public key, and the client verifies the certificate chain according to its trust configuration.

For example:

```
example.com
  ↓
Certificate
  ↓
Public Key
```

17. Very Important: TLS Does Not Mean "Encrypt Everything"

TLS protects the application data carried inside the secure connection.

Some metadata can still be visible to the network.

For example, depending on protocol/version and configuration, observers can generally see things such as:

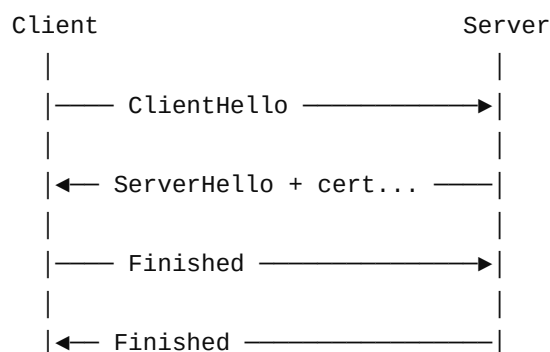
```
Source IP
Destination IP
Packet sizes/timing
```

HTTPS primarily protects the content of the HTTP communication.

18. TLS 1.3 — Simplified Flow

Modern TLS commonly uses TLS 1.3.

A simplified conceptual flow:



Don't memorize the exact packet structure yet.

For interviews, understand:

```
TCP
↓
TLS handshake
↓
Secure session
↓
HTTP
```

19. Step 7 — HTTP Request

Now the browser can send:

```
GET / HTTP/1.1
Host: example.com
```

Conceptually:

```
Browser
|
| HTTP GET
▼
TLS
|
▼
TCP
|
▼
IP
|
▼
Network
|
▼
Server
```

Because it's HTTPS, the HTTP request is carried inside the encrypted TLS connection.

20. What is GET?

GET is an HTTP method.

It generally means:

Retrieve a representation/resource.

Example:

```
GET /index.html HTTP/1.1
Host: example.com
```

21. Important HTTP Methods

Know these:

Method	Typical purpose
GET	Retrieve data
POST	Submit/create/process data
PUT	Replace/update a resource
PATCH	Partially update a resource
DELETE	Delete a resource
HEAD	Get headers without response body

For interviews, especially remember:

```
GET
POST
PUT
PATCH
DELETE
```

22. HTTP Request Structure

Conceptually:

Request Line



Headers



Blank line



Optional Body

Example:

```
POST /login HTTP/1.1
Host: example.com
Content-Type: application/json

{
  "username": "user"
}
```

23. HTTP Headers

Headers provide metadata.

Examples:

```
Host:
Content-Type:
Content-Length:
Authorization:
User-Agent:
Cookie:
Accept:
```

For example:

```
Content-Type: application/json
```

means the request/response body is JSON data.

24. Step 8 — Server Processes Request

The server receives:

```
Ethernet
↓
IP
↓
TCP
↓
TLS
↓
HTTP
```

Then the web server/application processes:

```
GET /
```

It might access:

```
Application
Database
Cache
File system
Other services
```

and create a response.

25. Step 9 — HTTP Response

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html
Content-Length: 1234

<html>
...
</html>
```

The response contains:

```
Status line
Headers
Body
```

26. HTTP Status Codes

Very important for interviews.

2xx — Success

200 OK
201 Created
204 No Content

3xx — Redirection

301 Moved Permanently
302 Found
304 Not Modified

4xx — Client-side error

400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found

5xx — Server-side error

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout

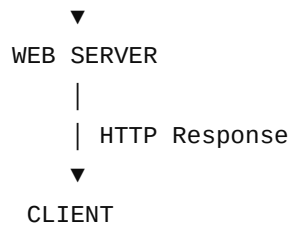
Memorize:

2xx → Success
3xx → Redirect
4xx → Client/request problem
5xx → Server-side problem

27. Complete Request/Response

CLIENT
|
| HTTP Request





Example:

```
GET /products
  ↓
200 OK
  ↓
JSON / HTML
```

28. Where Does TCP Segmentation Happen?

This connects directly to your previous learning.

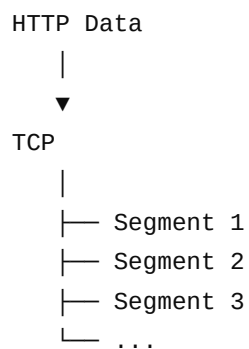
Suppose HTTP generates:

100 KB

of application data.

The TCP stack doesn't necessarily send that as one giant network packet.

Conceptually:

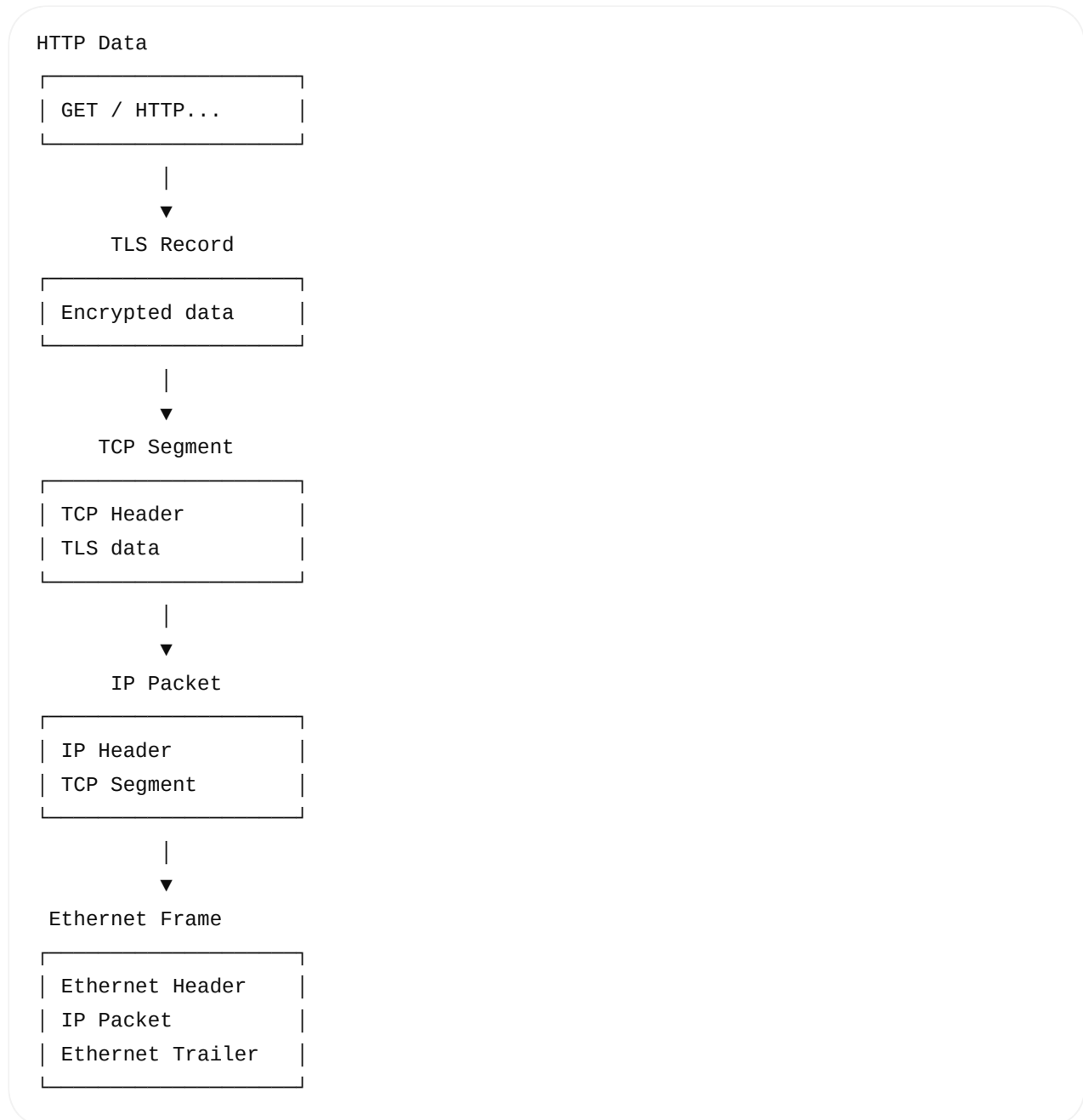


Then:

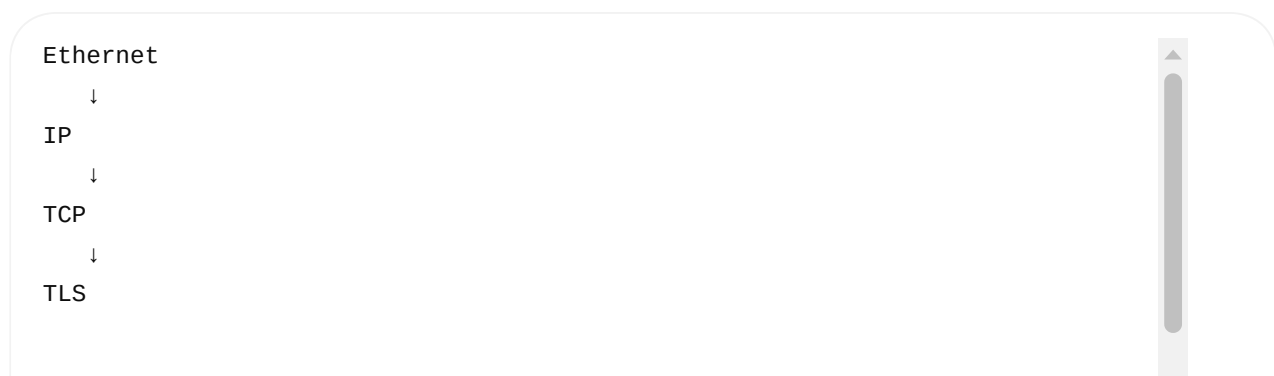
```
TCP Segment
  ↓
IP Packet
  ↓
Ethernet Frame
```

29. Complete Encapsulation

This is one of the most important diagrams from your networking preparation.



At the destination:



This is **decapsulation**.

30. Important Correction About Routers

Remember your previous question:

"Does the router only rework Layer 2?"

Let's apply it here.

Suppose:

PC A → Router → Router → Server

The router receives:

Ethernet Frame

It removes the incoming Layer-2 framing, examines the IP packet, performs routing, and creates a new Layer-2 frame for the next hop.

Conceptually:

Incoming Frame

↓

Remove L2

↓

Inspect IP

↓

Routing lookup

↓

New L2 header

↓

Outgoing Frame

The router normally does **not terminate the TCP connection** or decrypt HTTPS merely because it routes the packet.

31. What Changes at Every Router?

Suppose:

PC → R1 → R2 → Server

The IP packet generally continues toward the destination, although some IP header fields such as TTL are changed.

The Layer-2 frame is hop-specific.

PC → R1

Src MAC = PC

Dst MAC = R1

Then:

R1 → R2

Src MAC = R1

Dst MAC = R2

Then:

R2 → Server

Src MAC = R2

Dst MAC = Server

This is a very important concept.

32. What About TCP Ports?

Suppose browser connects to:

Server IP = 93.184.x.x

Server port = 443

The TCP packet contains:

Source port = ephemeral port

Destination port = 443

For example:

Source = 49152

Destination = 443

The exact client port varies.

So:

```
Client
192.168.1.10:49152
|
▼
Server
93.184.x.x:443
```

33. Why Does the Server Know Which Application Gets the Data?

Because of the destination port.

For example:

```
Destination port = 443
```

The operating system has a listening socket associated with that port.

Conceptually:

```
Incoming TCP
|
▼
Destination port 443
|
▼
Socket
|
▼
Web server
```

You can inspect listening TCP sockets with:

```
ss -ltn
```

34. curl — Your Best Hands-On Tool Today

Use:

```
curl https://example.com
```

This makes an HTTP(S) request from the command line.

Try:

```
curl -I https://example.com
```

`-I` requests headers only using a HEAD-style request.

You'll see something like:

```
HTTP/...  
content-type: text/html  
content-length: ...
```

35. See More Details with `curl -v`

Run:

```
curl -v https://example.com
```

This is extremely useful.

It can show information about:

```
DNS resolution  
TCP connection  
TLS handshake  
HTTP request  
HTTP response
```

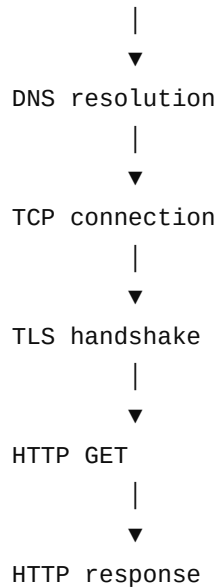
Conceptually:

```
DNS  
↓  
TCP  
↓  
TLS  
↓  
HTTP
```

This makes `curl -v` an excellent learning tool.

36. curl Visualization

```
curl https://example.com
```



37. Check TCP Connection with ss

While a connection is active, you can inspect sockets:

```
ss -tn
```

You might see:

```
ESTAB
192.168.1.10:49152
93.184.x.x:443
```

Meaning:

```
Local:
192.168.1.10:49152

Remote:
93.184.x.x:443

State:
ESTABLISHED
```

38. Capture Packets with tcpdump

If you have permission/root access, you can inspect traffic:

```
sudo tcpdump -i any
```

For TCP port 443:

```
sudo tcpdump -i any tcp port 443
```

You won't normally see the plaintext HTTP request inside HTTPS because TLS encrypts it.

But you can observe packet-level information such as:

```
IP addresses
TCP ports
TCP flags
packet sizes
sequence/acknowledgment information
```

39. TCP Flags You Should Recognize

During TCP connection establishment:

```
SYN
SYN, ACK
ACK
```

During normal communication:

```
ACK
```

Connection termination can involve:

```
FIN
ACK
```

or sometimes:

```
RST
```

40. HTTP Keep-Alive

Imagine the browser requests:

/index.html

Then:

/style.css

Then:

/script.js

Creating a new TCP connection for every object would be inefficient.

Modern HTTP supports connection reuse.

Conceptually:

TCP Connection

```
|  
├─ HTTP request 1  
├─ HTTP response 1  
├─ HTTP request 2  
├─ HTTP response 2  
└─ ...
```

This reduces connection setup overhead.

41. HTTP/1.1

HTTP/1.1 commonly uses persistent TCP connections.

Conceptually:

TCP

```
|  
├─ Request  
├─ Response  
├─ Request  
└─ Response
```

HTTP/1.1 also introduced important features such as:

Host header
Persistent connections
Chunked transfer encoding

42. HTTP/2

HTTP/2 still commonly uses:

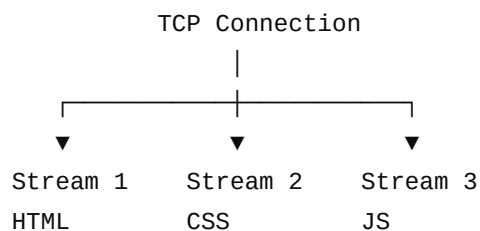
HTTP/2
↓
TLS
↓
TCP

But it improves application-layer efficiency.

One important concept:

Multiplexing

Multiple logical streams can share one TCP connection.



This is much more efficient than requiring separate connections for every resource.

43. HTTP/3

HTTP/3 changes the transport.

HTTP/1.1
↓
TCP

HTTP/2
↓
TCP

HTTP/3

↓
QUIC
↓
UDP

This is a very important interview point.

HTTP/3 does **not** use TCP as its transport.

QUIC provides transport features such as:

Reliable delivery
Encryption integration
Stream multiplexing
Connection migration

QUIC itself runs over UDP.

44. Compare HTTP Versions

Version	Transport	Important feature
HTTP/1.1	TCP	Persistent connections
HTTP/2	TCP	Multiplexed streams
HTTP/3	QUIC over UDP	Modern transport, stream multiplexing

For interviews:

HTTP/1.1 → TCP
HTTP/2 → TCP
HTTP/3 → QUIC/UDP

45. Full Website Flow — Interview Version

If interviewer asks:

"What happens when you type `https://google.com`?"

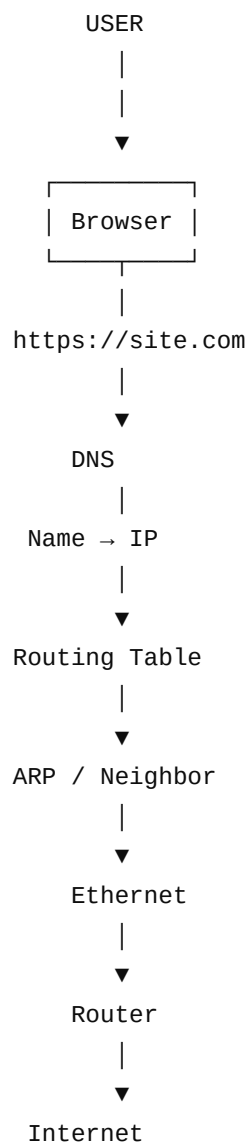
Give this structured answer:

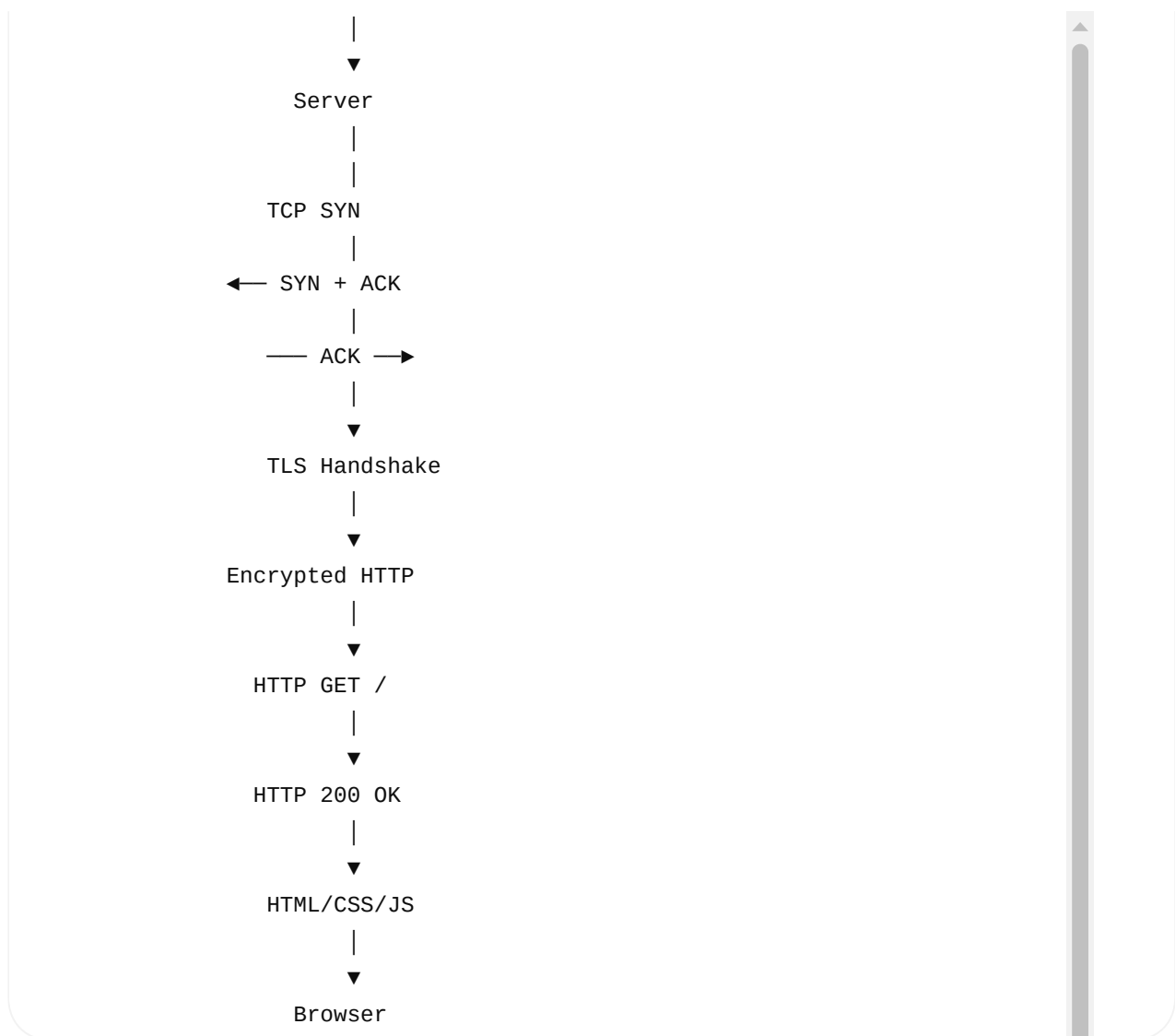
1. Browser parses URL
↓
2. DNS resolves hostname

- ↓
3. OS chooses route
 - ↓
 4. ARP/neighbor resolution finds next-hop MAC
 - ↓
 5. TCP 3-way handshake
 - ↓
 6. TLS handshake
 - ↓
 7. HTTP request
 - ↓
 8. Server processes request
 - ↓
 9. HTTP response
 - ↓
 10. Browser renders content

That's your **Day 12 master answer**.

46. Complete Visual Diagram





47. Encapsulation — Final Mental Model

When sending:

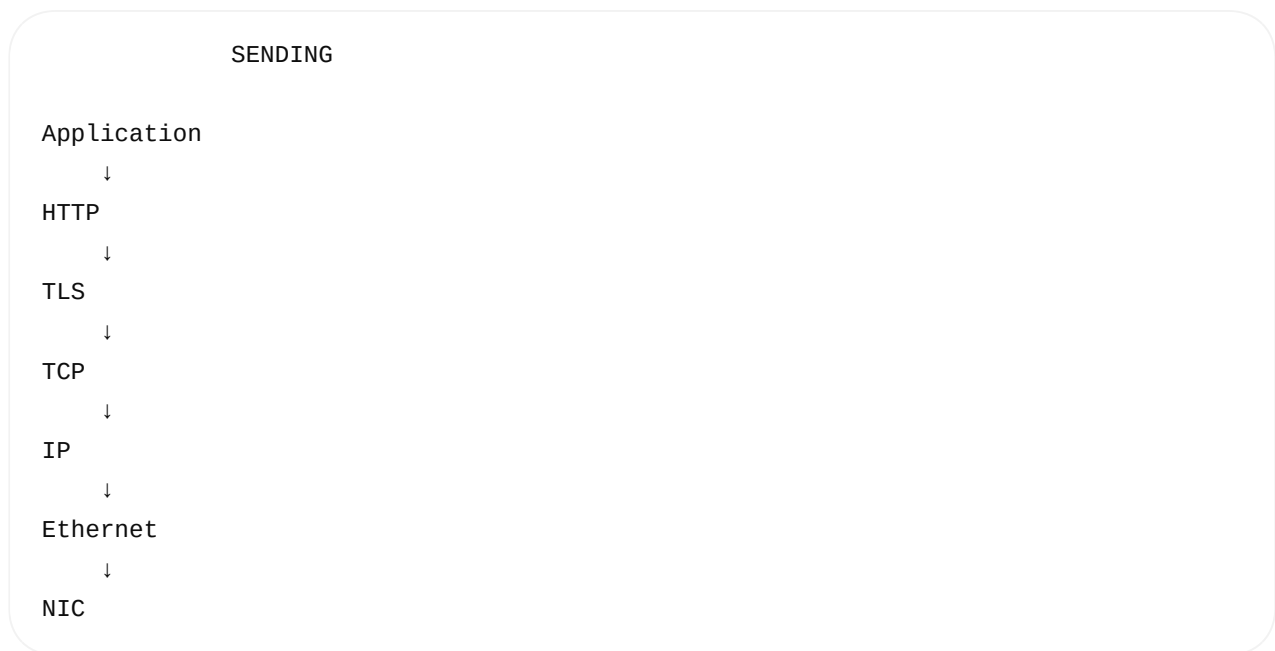
```
HTTP
↓
TLS
↓
TCP
↓
IP
↓
Ethernet
↓
NIC
```

At the receiving side:

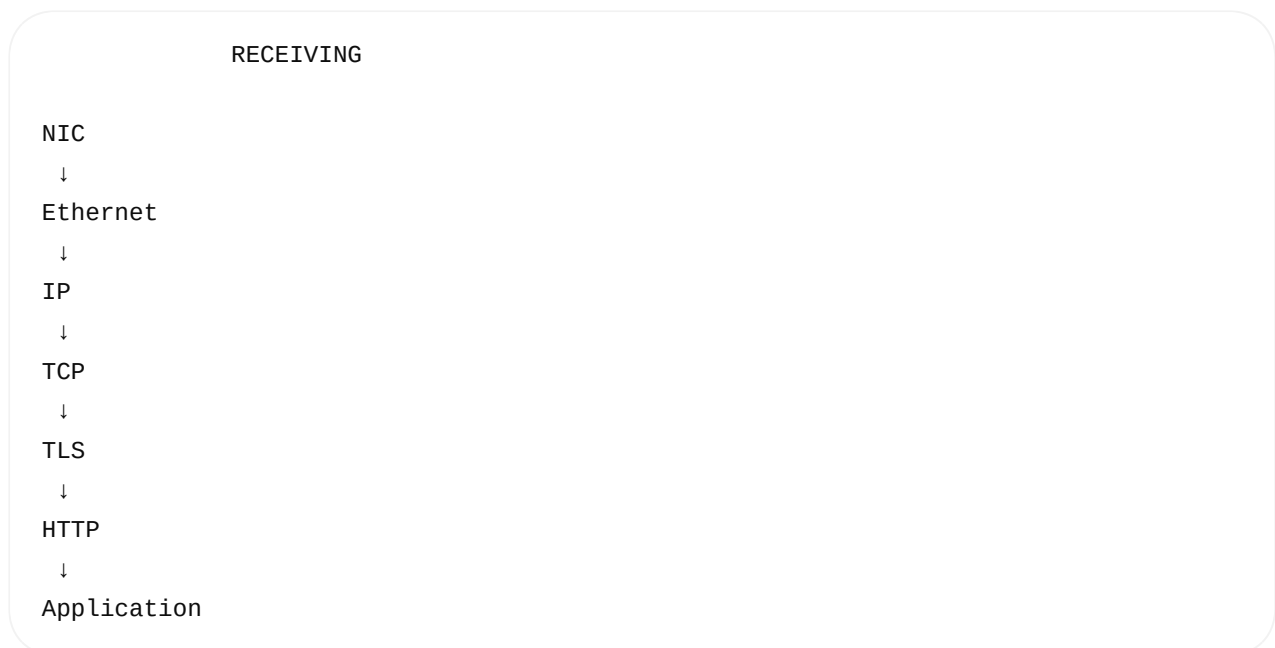
```
NIC
↓
```



So:



and:



Day 12 Hands-On — Ubuntu

Run these in order.

1. DNS

```
dig example.com
```

2. Routing

```
ip route get <IP_ADDRESS>
```

Replace <IP_ADDRESS> with the address returned by DNS.

3. HTTPS

```
curl -v https://example.com
```

Pay attention to:

```
Connected to ...  
SSL/TLS ...  
GET /  
HTTP/...
```

4. Headers

```
curl -I https://example.com
```

5. TCP sockets

```
ss -tn
```

6. Packet capture

```
sudo tcpdump -i any tcp port 443
```

Then in another terminal:

```
curl https://example.com
```

Observe the TCP packets.

★ Day 12 Interview Questions

Basic

1. What is HTTP?
2. What is HTTPS?
3. What is the difference between HTTP and HTTPS?
4. What port does HTTP normally use?
5. What port does HTTPS normally use?
6. What is an HTTP request?
7. What is an HTTP response?
8. What is GET?
9. What is POST?
10. What is an HTTP status code?

TCP + TLS

11. Does HTTPS use TCP?
12. Explain TCP 3-way handshake.
13. What happens after TCP handshake?
14. What is TLS?
15. Why is TLS required?
16. What is a certificate?
17. How does the browser verify the server?
18. Is HTTP data encrypted in HTTPS?
19. Does a router normally decrypt HTTPS traffic?

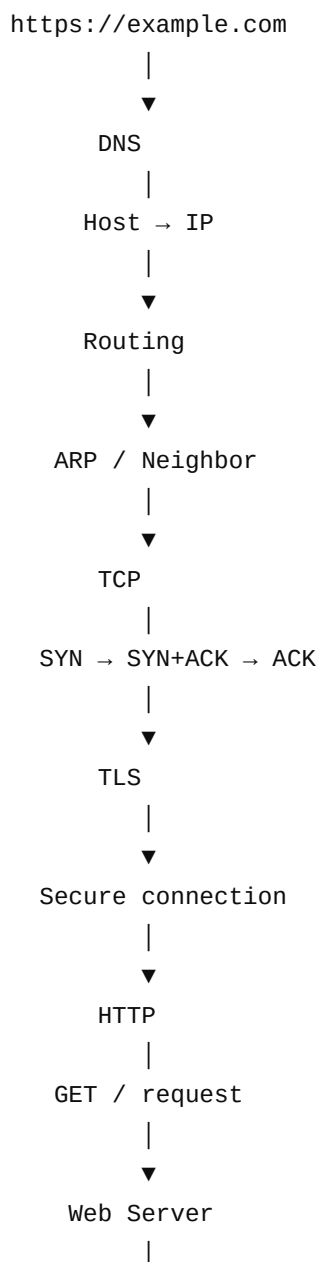
Deep

20. Explain what happens when you type `https://example.com`.
21. Where does DNS happen?

22. Where does routing happen?
 23. Where does ARP happen?
 24. Where does TCP segmentation happen?
 25. Where does TLS encryption happen?
 26. Where does HTTP processing happen?
 27. What changes at each router?
 28. HTTP/1.1 vs HTTP/2?
 29. HTTP/2 vs HTTP/3?
 30. Why does HTTP/3 use UDP?
-

Day 12 Must Memorize

If you remember only one diagram today, remember this:



200 OK response



Browser

The key relationship:

DNS	→ Finds WHERE
Routing	→ Finds HOW to reach
TCP	→ Provides reliable transport
TLS	→ Provides security
HTTP	→ Defines web communication
Ethernet	→ Moves frames on each local link
NIC	→ Sends/receives bits

Day 12 complete. Next in the 20-day schedule: Day 13 — Socket

Programming: `socket()`, `bind()`, `listen()`, `accept()`, `connect()`, `send()`, `recv()` and how these APIs map to TCP/IP.

Basic 1. What is HTTP? 2. What is HTTPS? 3. What is the difference between HTTP and HTTPS? 4. What port does HTTP normally use? 5. What port does HTTPS normally use? 6. What is an HTTP request? 7. What is an HTTP response? 8. What is GET? 9. What is POST? 10. What is an HTTP...

Answer

Absolutely. Here are the **Day 12 interview answers** in a beginner-friendly, interview-ready format. I'll also point out the **important words to say in an interview**.

Day 12 — Interview Answers

◆ Basic

1. What is HTTP?

HTTP = HyperText Transfer Protocol.

It is an **application-layer protocol** used for communication between a client, such as a browser, and a web server.

Example:

Browser



| HTTP Request





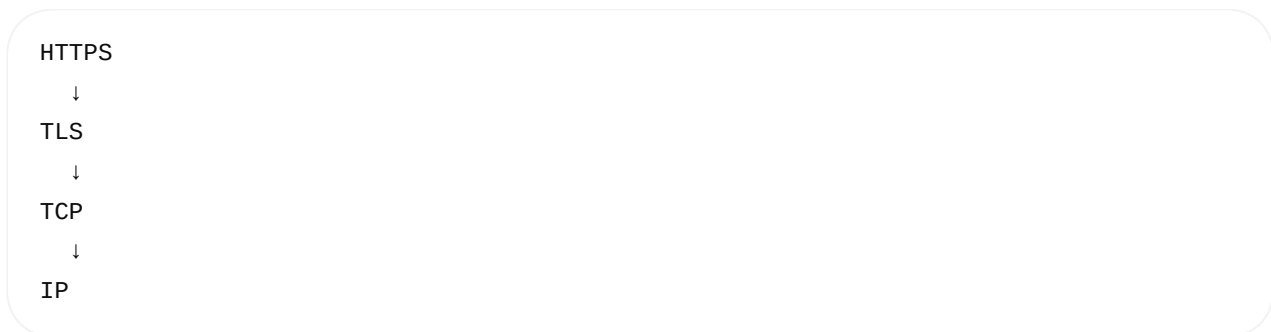
Interview answer:

HTTP is an application-layer protocol used to transfer resources such as HTML, JSON, images, CSS, and JavaScript between a client and a server.

2. What is HTTPS?

HTTPS = HTTP Secure.

It means HTTP communication is protected using **TLS (Transport Layer Security)**.



Interview answer:

HTTPS is HTTP communication protected by TLS, providing confidentiality, integrity, and server authentication.

3. Difference between HTTP and HTTPS

HTTP	HTTPS
No TLS protection	Uses TLS
Data is not protected by TLS	Application data is encrypted/protected
Normally port 80	Normally port 443
http://	https://

Simple:

HTTP

↓

TCP

HTTPS

↓

TLS

↓

TCP

4. What port does HTTP normally use?

Port 80.

HTTP → TCP port 80

Example:

Client: 49152

↓

Server: 80

5. What port does HTTPS normally use?

Port 443.

HTTPS → TCP port 443

For HTTP/3, HTTPS traffic uses **QUIC over UDP**, normally on UDP port 443.

6. What is an HTTP request?

An HTTP request is a message sent by the client to the server asking it to perform an operation, commonly to retrieve or modify a resource.

Example:

GET /index.html HTTP/1.1

Host: example.com

Structure:

```
Request line
  ↓
Headers
  ↓
Optional body
```

7. What is an HTTP response?

An HTTP response is the message sent by the server back to the client.

Example:

```
HTTP/1.1 200 OK
Content-Type: text/html

<html>
...
</html>
```

Structure:

```
Status line
  ↓
Headers
  ↓
Optional body
```

8. What is GET?

GET is an HTTP method generally used to **retrieve a resource**.

Example:

```
GET /products HTTP/1.1
Host: example.com
```

Meaning:

"Give me the /products resource."

9. What is POST?

POST is commonly used to **submit data to the server**, often to create a resource or trigger server-side processing.

Example:

```
POST /login HTTP/1.1
Content-Type: application/json

{
  "username": "user"
}
```

Simple difference:

```
GET → retrieve
POST → submit/process data
```

10. What is an HTTP status code?

An HTTP status code tells the client the result of the request.

2xx — Success

```
200 OK
201 Created
204 No Content
```

3xx — Redirection

```
301
302
304
```

4xx — Client/request error

```
400 Bad Request
401 Unauthorized
403 Forbidden
404 Not Found
```

5xx — Server error

500 Internal Server Error
502 Bad Gateway
503 Service Unavailable
504 Gateway Timeout

Remember:

2xx → Success
3xx → Redirect
4xx → Client/request problem
5xx → Server problem

◆ TCP + TLS

11. Does HTTPS use TCP?

For traditional HTTP/1.1 and HTTP/2:

Yes.

HTTP/HTTPS
↓
TLS
↓
TCP
↓
IP

But HTTP/3 is different:

HTTP/3
↓
QUIC
↓
UDP

Interview answer:

HTTPS commonly runs over TLS over TCP for HTTP/1.1 and HTTP/2. HTTP/3 uses QUIC over UDP.

12. Explain TCP 3-way handshake

The TCP connection is established using three messages.

1. SYN

Client → Server

Client — SYN → Server

Client says:

I want to establish a TCP connection.

2. SYN + ACK

Server → Client

Client ← SYN + ACK — Server

Server says:

I received your request and I am also ready.

3. ACK

Client → Server

Client — ACK → Server

Client confirms:

I received your response.

Now:

TCP connection established

Remember:

SYN
↓
SYN + ACK
↓
ACK

13. What happens after TCP handshake?

For HTTPS, the next major step is the **TLS handshake**.

```
TCP handshake
  ↓
TCP connection established
  ↓
TLS handshake
  ↓
Secure TLS connection
  ↓
HTTP request
```

So:

```
SYN
  ↓
SYN+ACK
  ↓
ACK
  ↓
TLS
  ↓
HTTP GET
```

14. What is TLS?

TLS = Transport Layer Security.

TLS is a cryptographic protocol that provides security for network communication.

It provides:

```
Confidentiality
Integrity
Authentication
```

Conceptually:

```
HTTP data
  ↓
TLS protection
  ↓
TCP
```



15. Why is TLS required?

TLS is used to protect communication from eavesdropping and tampering and to authenticate the server.

Without HTTPS:

```
graph TD; A[HTTP data] --> B[Network];
```

HTTP data
↓
Network

With HTTPS:

```
graph TD; A[HTTP data] --> B[TLS protection]; B --> C[Network];
```

HTTP data
↓
TLS protection
↓
Network

Important:

TLS protects the application data carried by the secure connection.

16. What is a certificate?

A TLS certificate is used to bind a server identity, such as a domain name, to a public key.

Conceptually:

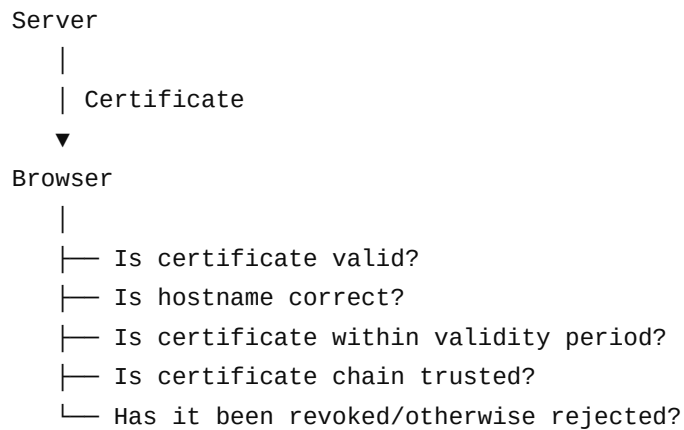
```
graph TD; A[example.com] --> B[TLS Certificate]; B --> C[Public Key];
```

example.com
↓
TLS Certificate
↓
Public Key

The certificate is digitally signed by a trusted certificate authority, directly or through an intermediate CA.

17. How does the browser verify the server?

Simplified process:



If validation succeeds, the browser can establish trust in the server identity.

Interview answer:

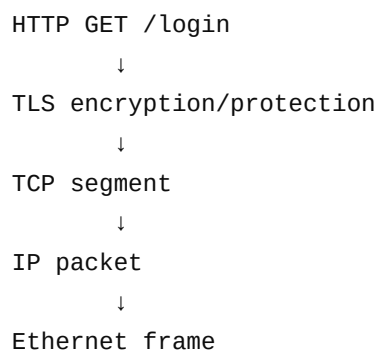
The browser validates the certificate chain against trusted CAs, checks the hostname and certificate validity, and verifies the cryptographic signatures.

18. Is HTTP data encrypted in HTTPS?

Yes.

After the TLS handshake, HTTP application data is protected by TLS.

Conceptually:



A network observer cannot normally read the HTTP request/response contents.

But metadata such as:

Source IP
Destination IP

can still be observable.

19. Does a router normally decrypt HTTPS traffic?

No.

A normal IP router forwards packets.

```
Client
  ↓
Router
  ↓
Router
  ↓
Server
```

The router normally sees network-layer information such as:

```
Source IP
Destination IP
```

and forwards the packet.

It does **not normally decrypt the HTTPS application data**.

Important exception:

A security gateway/proxy/load balancer can intentionally terminate TLS if configured to do so. That's different from ordinary IP routing.

Deep Questions

20. Explain what happens when you type `https://example.com`

This is one of the **most important networking interview questions**.

Answer in this order:

Step 1 — Browser parses URL

```
https://example.com
```

The browser identifies:

```
Protocol = HTTPS
Host      = example.com
Port      = 443
Path      = /
```

Step 2 — DNS

```
example.com
  ↓
DNS
  ↓
IP address
```

Step 3 — Routing

Linux checks:

```
ip route
```

and determines the appropriate interface/next hop.

Step 4 — ARP / Neighbor discovery

For an IPv4 Ethernet next hop:

```
Next-hop IP
  ↓
ARP
  ↓
Next-hop MAC
```

Step 5 — TCP handshake

```
SYN
  ↓
SYN+ACK
  ↓
ACK
```

Step 6 — TLS handshake

```
ClientHello
  ↓
ServerHello + certificate...
  ↓
```



Key establishment



Secure session

Step 7 — HTTP request

GET / HTTP/1.1

Host: example.com

Step 8 — Server response

HTTP/1.1 200 OK

Complete answer:

URL



DNS



IP address



Routing



ARP/Neighbor



TCP handshake



TLS handshake



HTTP request



Server processing



HTTP response



Browser rendering

★ This flow is worth memorizing.

21. Where does DNS happen?

DNS is an **application-layer** operation.

Application



DNS

The DNS query itself then travels through the lower layers:

DNS



UDP/TCP



IP



Ethernet/Wi-Fi



NIC

22. Where does routing happen?

Routing is primarily a **Layer 3 / IP** function.

On Linux, the kernel uses the routing table to determine the next hop/interface.

```
ip route
```

Example:

Destination



Routing table



Next hop



Interface

Routers perform Layer 3 forwarding as well.

23. Where does ARP happen?

ARP maps:

IPv4 address



MAC address

for communication on a local Ethernet-type network.

Example:

192.168.1.1
↓
ARP
↓
AA:BB:CC:DD:EE:FF

ARP is associated with the **Layer 2/Layer 3 boundary**.

In an interview, say:

ARP resolves an IPv4 next-hop address to a MAC address on the local network.

Don't simply say:

"ARP is Layer 2."

The exact OSI classification is often described as between Layers 2 and 3.

24. Where does TCP segmentation happen?

TCP operates at **Layer 4 / Transport Layer**.

Suppose the application provides:

100 KB

TCP may divide the byte stream into segments according to the current TCP conditions.

Application data
↓
TCP
├───┴───┬───┴───┬───┴───┬───
↓ ↓ ↓
Seg1 Seg2 Seg3

Then each segment is passed to IP:

TCP Segment
↓
IP Packet

25. Where does TLS encryption happen?

TLS sits between the application protocol and transport in the traditional HTTPS stack.

```
graph TD; HTTP --> TLS; TLS --> TCP;
```

HTTP
↓
TLS
↓
TCP

The HTTP data is protected by TLS before being passed down to TCP.

```
graph TD; HTTP_data[HTTP data] --> TLS; TLS --> Protected_TLS[Protected TLS records]; Protected_TLS --> TCP;
```

HTTP data
↓
TLS
↓
Protected TLS records
↓
TCP

26. Where does HTTP processing happen?

HTTP is an **application-layer protocol**.

The browser/client creates HTTP requests.

The web server/application processes them.

```
graph TD; Browser1[Browser] -- "HTTP Request" --> Web_Server[Web Server]; Web_Server -- "HTTP Response" --> Browser2[Browser];
```

Browser
|
| HTTP Request
▼
Web Server
|
| HTTP Response
▼
Browser

27. What changes at each router?

This is very important.

Suppose:

PC A → Router 1 → Router 2 → Server

At each router:

Layer 2 changes

The Ethernet frame is removed and a new Layer-2 frame is created for the next hop.

PC → R1

MAC A → MAC R1

Then:

R1 → R2

MAC R1 → MAC R2

Then:

R2 → Server

MAC R2 → MAC Server

IP packet

The destination IP generally remains the same as the packet travels toward the destination.

Some IP fields, such as TTL, change.

TCP

Normally remains end-to-end.

HTTP/TLS

Normally remains end-to-end.

So:

MAC	→ changes every hop
IP	→ generally end-to-end destination/source
TCP	→ end-to-end
TLS	→ end-to-end
HTTP	→ end-to-end

There are exceptions such as NAT, proxies, load balancers, and TLS-terminating middleboxes.

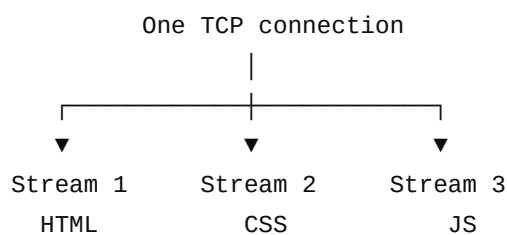
28. HTTP/1.1 vs HTTP/2

HTTP/1.1	HTTP/2
TCP	TCP
Text-based message format	Binary framing
Persistent connections	Persistent connections
Limited request concurrency per connection	Multiplexed streams
Head-of-line issues at HTTP level	Better multiplexing

The most important point:

```
graph TD
    A[HTTP/1.1] --> B[TCP]
    C[HTTP/2] --> D["TCP + multiplexing"]
```

HTTP/2 visualization



29. HTTP/2 vs HTTP/3

This is another important interview question.

HTTP/2

```
graph TD
    A[HTTP/2] --> B[TCP]
    B --> C[IP]
```

HTTP/3

```
HTTP/3
  ↓
QUIC
  ↓
UDP
  ↓
IP
```

Main difference:

```
HTTP/2 → TCP
HTTP/3 → QUIC/UDP
```

QUIC provides reliable, encrypted, multiplexed transport functionality above UDP.

30. Why does HTTP/3 use UDP?

This is a **very good interview question**.

A common beginner answer is:

"UDP is faster than TCP."

That's too simplistic.

Better answer:

HTTP/3 uses QUIC, which runs over UDP. QUIC implements transport features such as reliability, congestion control, encryption integration, and independent streams in user space, allowing modern connection establishment and stream handling without depending on TCP.

Visualize:

```
HTTP/2

HTTP
  ↓
TCP
  ↓
IP
```

versus:

HTTP/3

HTTP

↓

QUIC

↓

UDP

↓

IP

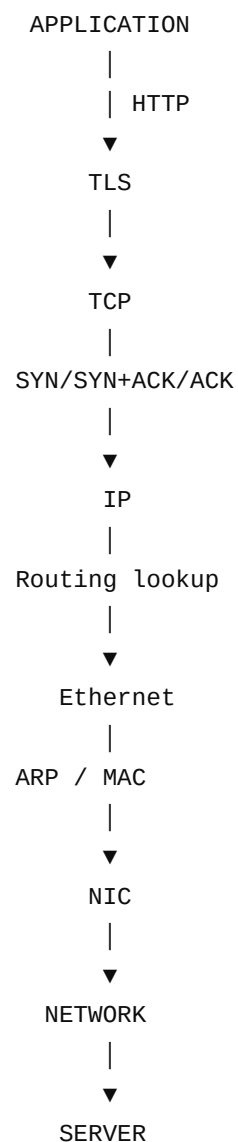
One important benefit is reducing some limitations associated with TCP's transport-level head-of-line blocking.

★ Final Day 12 Revision Table

Concept	Answer
HTTP	Application-layer web protocol
HTTPS	HTTP protected by TLS
HTTP port	80
HTTPS port	443
DNS	Hostname → IP
Routing	Determines next hop/interface
ARP	IPv4 next-hop IP → MAC
TCP	Reliable transport
TCP handshake	SYN → SYN+ACK → ACK
TLS	Security/authentication/integrity/confidentiality
HTTP GET	Retrieve resource
HTTP POST	Submit/process data
2xx	Success
3xx	Redirection

Concept	Answer
4xx	Client/request error
5xx	Server error
HTTP/1.1	TCP
HTTP/2	TCP + multiplexing
HTTP/3	QUIC over UDP

One diagram to memorize



Most important interview sequence:

DNS → Routing → ARP → TCP handshake → TLS handshake → HTTP request → HTTP response

That sequence connects almost everything you've learned from **Day 1 through Day 12**.